Java IO

Instructor:

©FPT SOFTWARE - Corporate Training Center - Internal Use

Table of contents

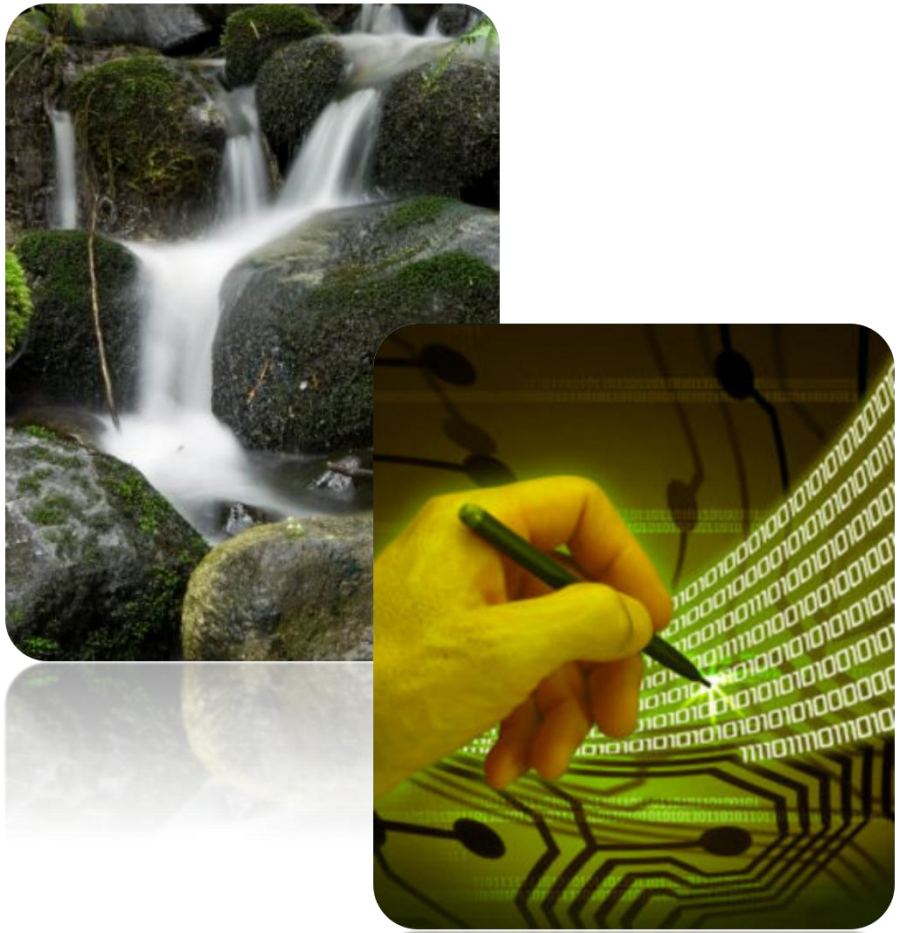
Learning Approach

Section 1

JAVA IO INTRODUCTION

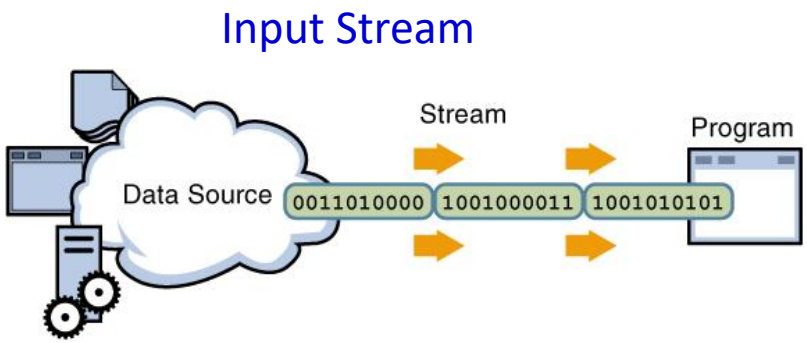
Java I/O Introduction

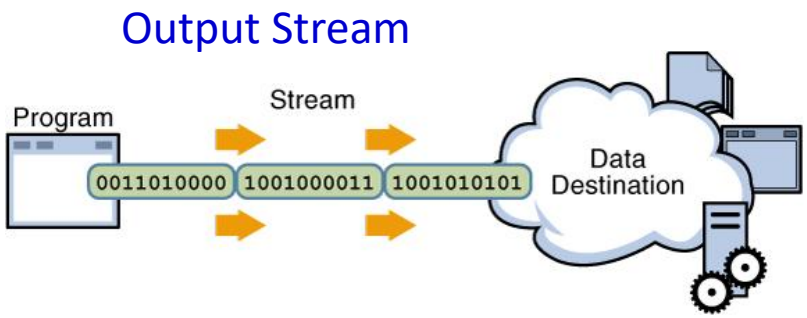
- Java Input Output(IO) is one of the most important topics in Java.



What is a Stream?

- A stream is an ordered sequence of bytes of indeterminate length.
- A data stream is a channel through which data travels from a source to a destination
- There are two type of stream:





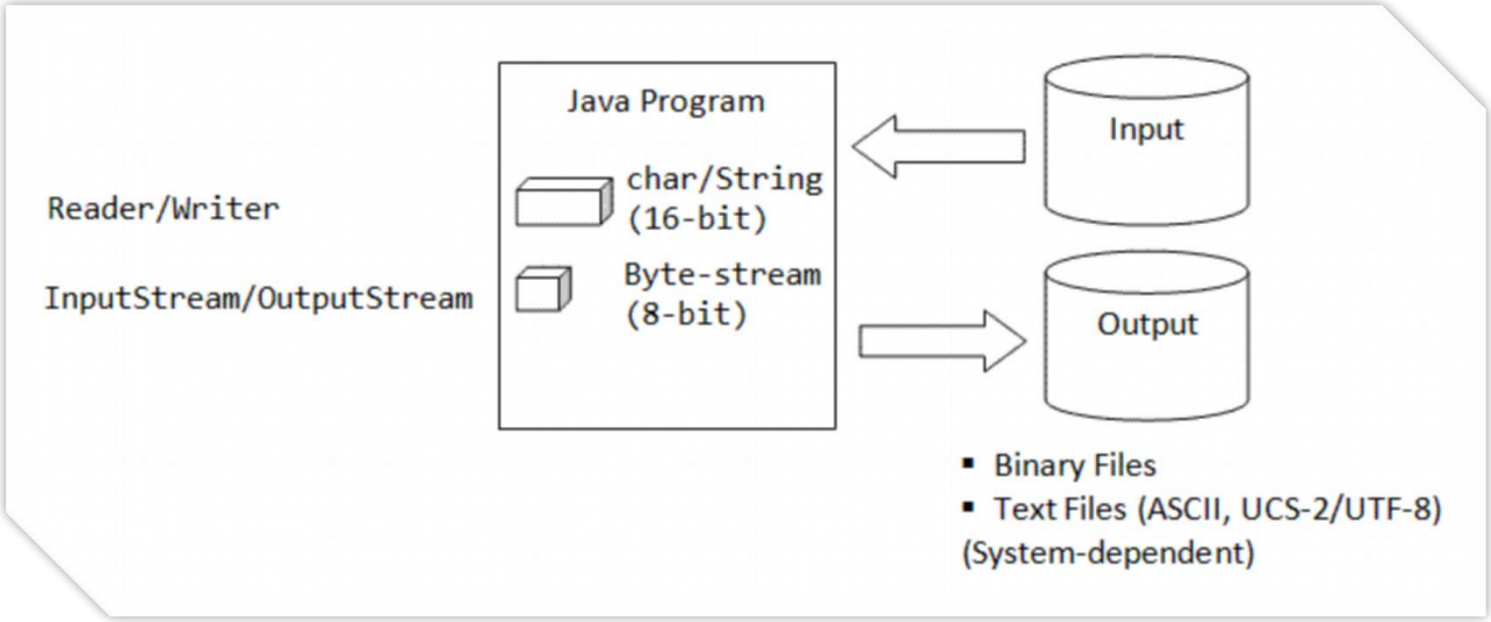
I/O Operations

I/O operations involve three steps:

1. **Open** an input/output stream associated with a physical device (e.g., file, network, console/keyboard), by constructing an appropriate IO-stream object.
2. **Read** from the opened input stream until "end-of-stream" encountered, or **Write** to the opened output stream.
3. **Close** the input/output stream.

Types of I/O data stream

- Java uses 16-bit character set (instead of 8-bit character set).
- As a consequence, it needs to differentiate between *byte-based I/O* and *character-based I/O*.



Section 1

JAVA IO: FILE

©FPT SOFTWARE - Corporate Training Center - Internal Use

Introduction

- File class directly works with files and directories.
- The **File** only gives you access to the file and file system meta data. If you need to read or write the content of files, you should do so using either ***FileInputStream***, ***FileOutputStream*** or ***RandomAccessFile***.
- Using the **File** class you can:
  - ✓ Check if a file or directory exists.
  - ✓ Create a directory if it does not exist.
  - ✓ Read the length of a file.
  - ✓ Rename or move a file.
  - ✓ Delete a file.
  - ✓ Check if path is file or directory.
  - ✓ Read list of files in a directory.

Instantiating a java.io.File

- Before you can do anything with the file system or File class, you must obtain a File instance.
- Here is how that is done:

```
File file = new File("D:\\Documents\\input-file.txt");
```

```
File file = new File(new File("D:\\Documents",
            "Homework"), "data.txt");
```

Check if File Exists

- Once you have instantiated a **File** object you can check if the corresponding file actually exists already.
- The **File** class constructor will not fail if the file does not already exists.
- To check if the file exists, call the **exists()** method. Here is a simple example:

```
boolean fileExists = file.exists();
```

Create a Directory

- The File class contains the method **mkdir()** and **mkdirs()** to create a directory.
- The mkdir() method creates a single directory if it does not already exist. Here is an example:

```
File file = new
                File("D:\\Documents\\Download\\Collection");
boolean dirCreated = file.mkdir();
```

  - ✓ Provided that the directory **D:\Documents\Download** already exists, the above code will create a subdirectory of **Download** named **Collection**.
  - ✓ The **mkdir()** returns **true** if the directory was created, and **false** if not.
- The mkdirs() will create all directories that are missing in the path the File object represents. Here is an example:

```
boolean dirCreated = file.mkdirs();
```

  - ✓ will create all the directories in the path **c:\users\jakobjenkov\newdir**. The mkdirs() method will return true if all the directories were created, and false if not.

Delete a File

```
public class FileDelete {
    public static void main(String[] args) throws IOException {
        File file = null;
        boolean deleted = false;
        String message = null;

        file = new File("filename.txt");
        deleted = file.delete();
        message = (deleted) ? "Deleted file" : "File is not exists";
        System.out.println(message);
        /* Create file */
        file.createNewFile();
        System.out.println("Call create new file");
        deleted = file.delete();
        message = (deleted) ? "Deleted file" : "File is not exists";
        System.out.println(message);
    }
}
```

Delete a Folder

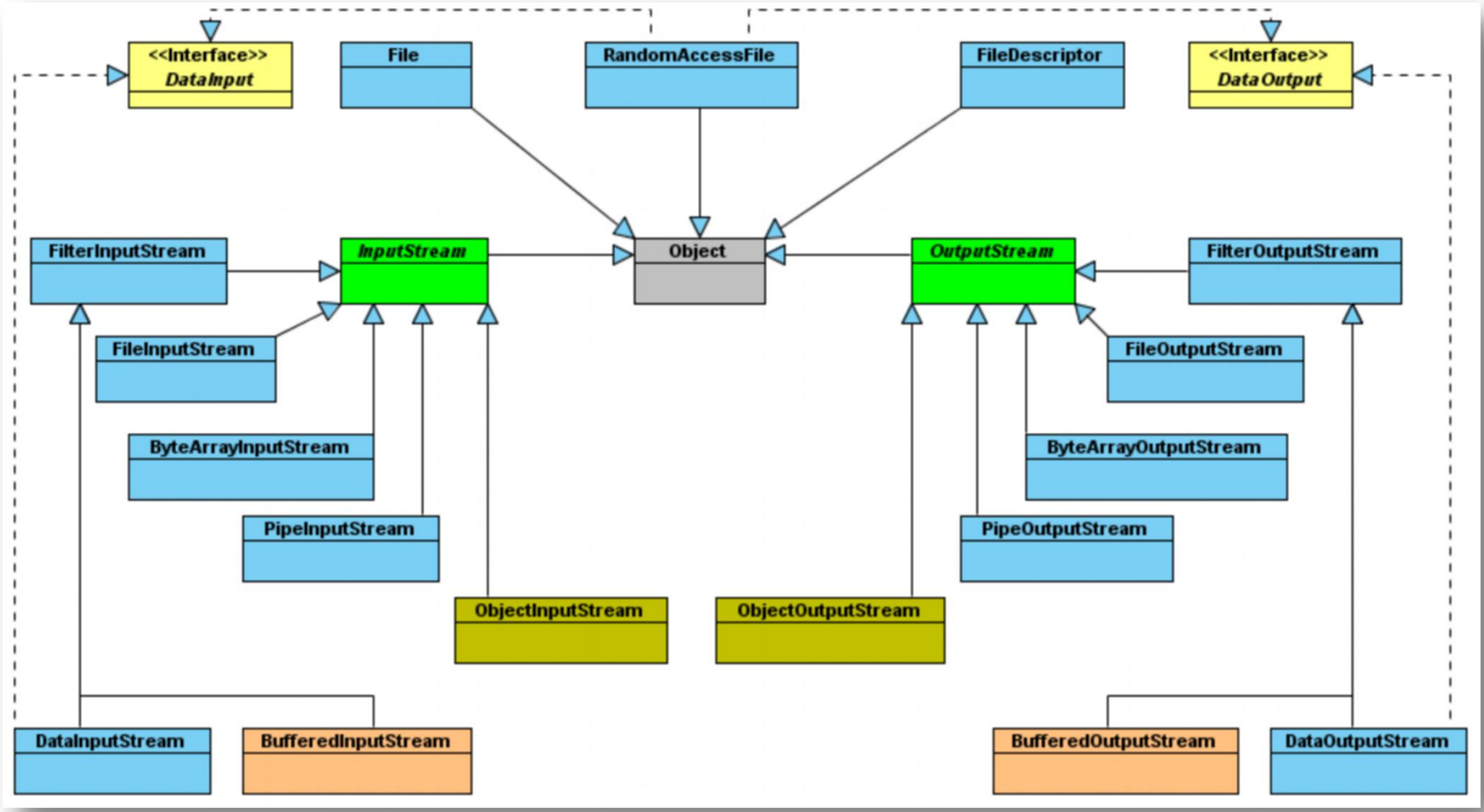
```
public class FolderDelete {
    public static void main(String[] args) throws IOException {
        File file1 = new File("D:\\Documents\\Download");
        file1.mkdirs();
        File file2 = new File(file1, "Collection");
        file2.mkdir();
        File file3 = new File(file1, "Data");
        file3.createNewFile();

        /*
         * Khong the xoa thu muc neu no KHONG rong
         */
        boolean del = file1.delete();
        System.out.println("Deleted parent folder? " + del);

        del = file2.delete();
        System.out.println("Deleted sub-folder? " + del);
    }
}
```

Section 2

BINARY STREAMS

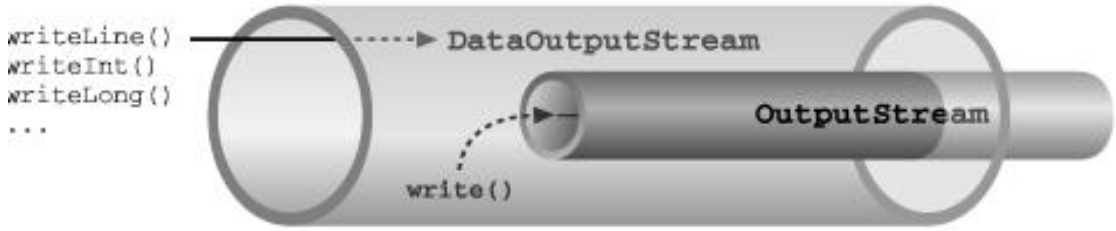
java.io Package

Byte-Based IO & Byte Streams



DataOutput Interface

- Convert data present in Java primitive type into a series of bytes and write them onto a binary stream
- Convert string data from into Java modified UTF-8 format and write it into a stream
- Methods:
  - **writeBoolean()**
  - **writeByte()**
  - **writeInt()**
  - **writeDouble()**
  - **writeChar()**
  - **writeChars()** and **writeUTF()**



```java
public class Person {
    private String name;
    private byte age;
    private String nationality;

    public Person() {

    /**
     *
     * @param name
     * @param age
     * @param nationality
     */

    public Person(String name, byte age, String nationality) {

    public String getName() {

    public void setName(String name) {

    public byte getAge() {

    public void setAge(byte age) {

    public String getNationality() {
```

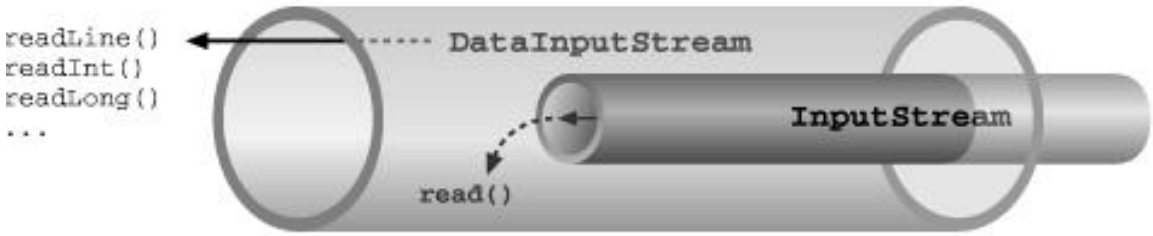
```java
public class Coach extends Person {
    private byte year;

    public byte getYear() {

    public void setYear(byte year) {

    public Coach(String name, byte age, String nationality, byte year) {

    public Coach() {

    public void display() {
```

DataOutput Example

```
public class DataOutputStreamExample {
    public static void main(String[] args) throws IOException {
        Coach[] coachs = new Coach[3];
        coachs[0]= new Coach("John", (byte)30, "Vietnam", (byte) 5);
        coachs[1] = new Coach ("Keen Luke", (byte)32, "Vietnam", (byte) 8);
        coachs[2] = new Coach ("Tabor Fred", (byte)28, "Vietnam", (byte) 3);
        //
        // Create FileOutputStream write to file.
        //
        FileOutputStream fos = new FileOutputStream("D:/Documents/Data/coach.txt", true);

        // Create DataOutputStream object wrap 'fos'.
        // The data write to 'dos' will be pushed to 'fos'.
        DataOutputStream dos = new DataOutputStream(fos);
        // Write data.
        for (Coach coach: coachs) {
            dos.writeUTF(coach.getName());
            dos.writeByte(coach.getAge());
            dos.writeUTF(coach.getNationality());
            dos.writeByte(coach.getYear());
        }
        dos.flush();
        dos.close();
    }
}
```

DataInput inteface

- Reading bytes from a binary stream and convert the data to any of the Java primitive types
- Convert data from Java modified Unicode Transmission Format format into string form
- Methods:
  - **readBoolean()**
  - **readByte()**
  - **readInt()**
  - **readDouble()**
  - **readChar()**
  - **readLine()** and **readUTF()**



DataInput Example

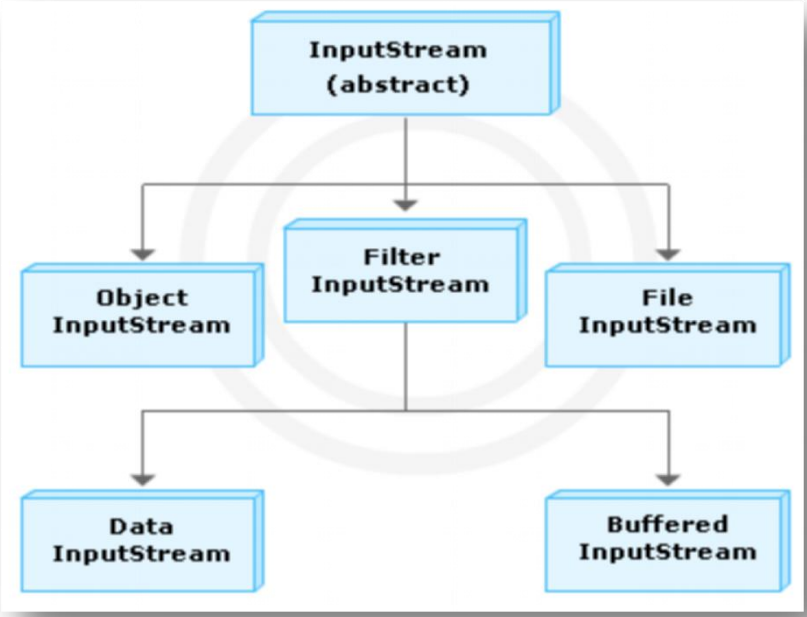
```
public class DataInputStreamExample {
	public static void main(String[] args) throws IOException {
		boolean EOF = false;
		// Stream to read file.
		FileInputStream fis = new FileInputStream("D:/Documents/Data/coach.txt");
		// Create DataInputStream object wrap 'fis'.
		DataInputStream dis = new DataInputStream(fis);
		// Read data.
		String name;
		byte age;
		String nationality;
		byte year;
		while (!EOF) {
			try {
			name = dis.readUTF();
			System.out.println("Name: " + name);
			age = dis.readByte();
			System.out.println("Age: " + age);
			nationality = dis.readUTF();
			System.out.println("Nationality: " + nationality);
			year = dis.readByte();
			System.out.println("Year: " + year);
			} catch (EOFException e) {
			  EOF = true;
			}
		}
		dis.close();
	}
}
```

InputStream class

- InputStream class is an abstract class that defines how streams receive data and is the superclass of all stream classes
- Methods: **read(), available(), close(), mark(), skip()** and **reset().**

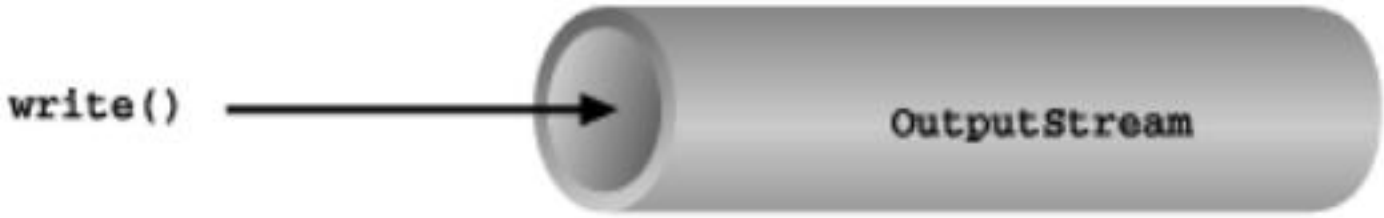
InputStream sub-classes

- **FileInputStream** class is used to read bytes from a file.
- **FileInputStream** class overrides all the methods of the InputStream class except mark() and reset().
- **ByteArrayInputStream** class contains a buffer (a byte array) that stores the bytes that are read from the stream.



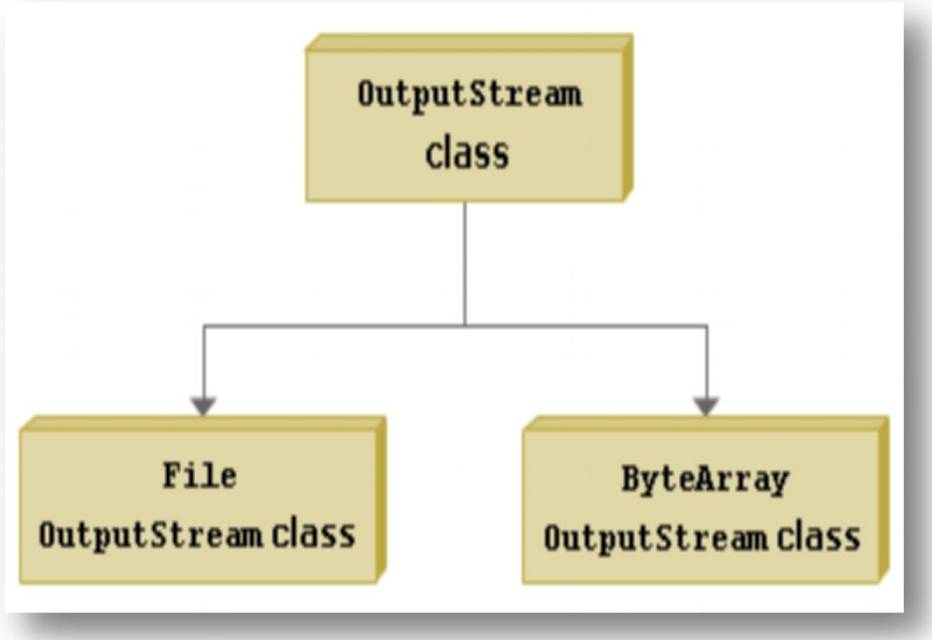
OuputStream class

- **OutputStream** class is an abstract class that defines the method in which bytes or array of bytes are written to streams
- Methods: **write()**, **flush()**, **close()…**



OutputStream sub-classes

- **FileOutputstream** class creates an OutputStream that is used to write bytes to a file.
- **ByteArrayOutputStream** class creates an output stream in which the data is written using a byte array.



File IO Streams

FileInputStream & FileOuputStream

Open a file:

```
File file = new File("filename");
FileInputStream in=new FileInputStream(file);
```

Read data:

```
int byteRead=0;
while (byteRead!=-1) {
          byteRead=in.read();
}
```

Save data to a file:

```
File file = new File("filename");
FileOutputStream out=new FileOutputStream(file);
```

```
int byteRead=0;
{
          byteRead=in.read();
          out.write(byteRead);
}while (byteRead!=-1)
```

Close all:

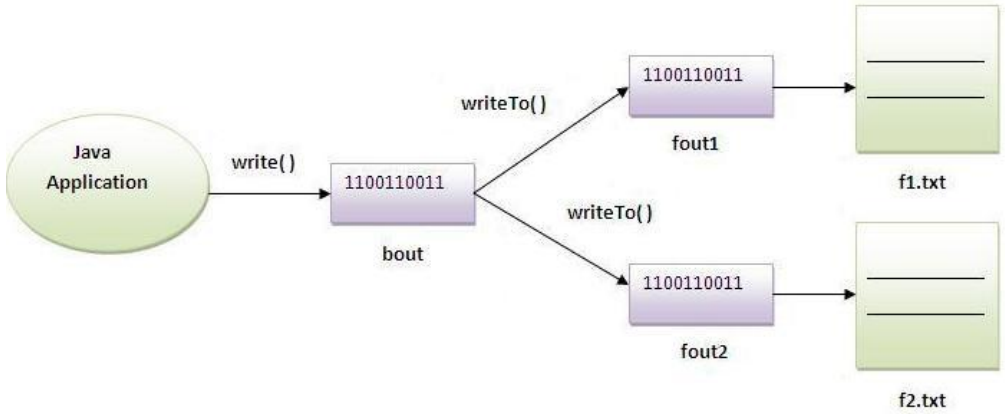
```
try{
//Do Open, Read, Write data
}catch(IOException ex) {
    ex.printStackTrace();
}finally {
    try {
          if (in != null) in.close();
          if (out != null) out.close();
    } catch (IOException ex) {
         ex.printStackTrace();
    }
}
```

Example 1

```
public class ByteArrayStreamExample {
    public static void main(String[] args) throws IOException {
        // Create ByteArrayOutputStream object.
        // Object contains within it an array of bytes. Array with size 12 elements.
        // If the number of elements to write to stream more than 12, the array will be replaced by
        // new array has more elements, and copy the elements of old array into.
        ByteArrayOutputStream os = new ByteArrayOutputStream(12);
        String sData = "Hello FPT";
        byte[] bData = sData.getBytes();
        os.write(bData);

        // Returns the current size of the buffer.
        int size = os.size();
        System.out.println("Size = " + size);
        byte[] bInputData = new byte[size];

        // Using ByteArrayInputStream to read bytes array.
        ByteArrayInputStream is = new ByteArrayInputStream(bData);
        is.read(bInputData);
        for (byte b : bInputData) {
        System.out.print((char) b + " ");
        }
        os.flush();
        os.close();
    }
}
```

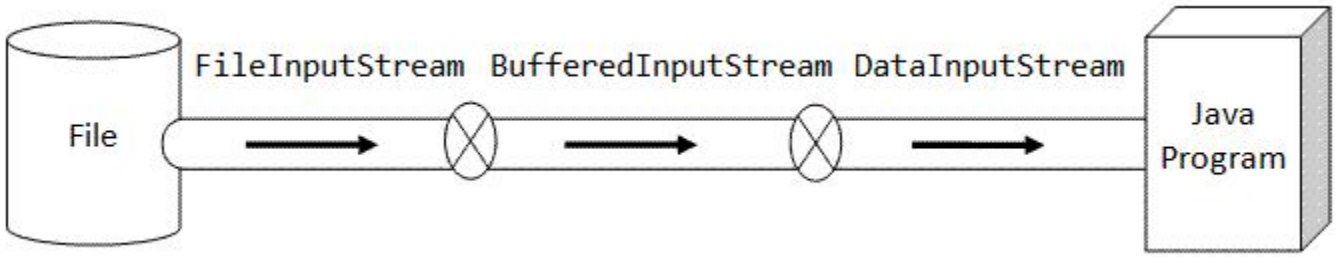
Example 2

```
public class FileOutpt_ByteArrayExample {
    public static void main(String args[]) throws Exception {
        FileOutputStream fout1 = new FileOutputStream("D:\\Documents\\Data\\f1.txt");
        FileOutputStream fout2 = new FileOutputStream("D:\\Documents\\Data\\f2.txt");

        ByteArrayOutputStream bout = new ByteArrayOutputStream();
        bout.write(139);
        bout.writeTo(fout1);
        bout.writeTo(fout2);

        bout.flush();
        bout.close();// has no effect
        System.out.println("success...");
    }
}
```



Buffered I/O

- A buffer is a **temporary storage area** for data.
- By storing the data in a buffer, time is saved as data is immediately received from the buffer instead of going back to the original source of the data.

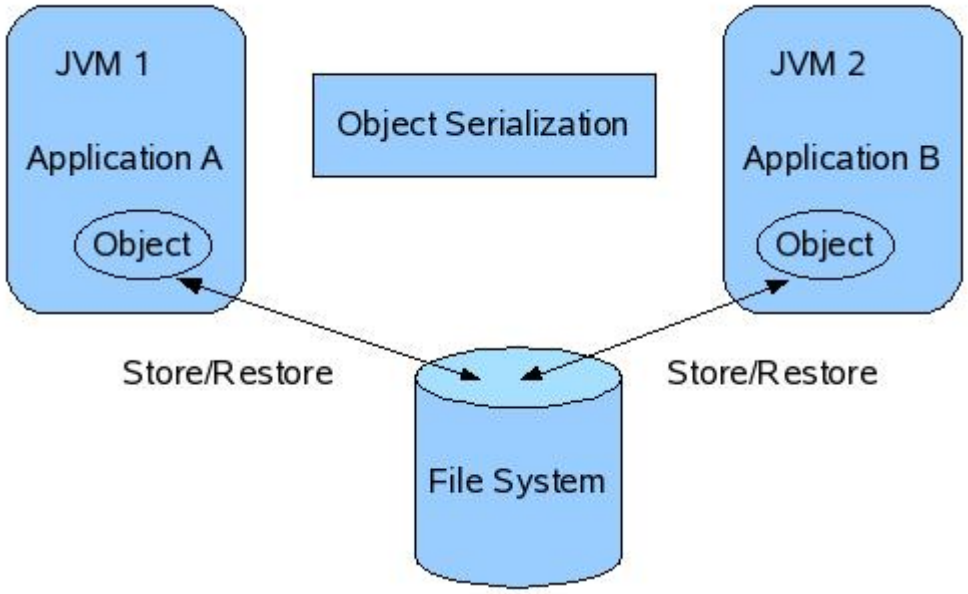



```
FileInputStream fin = new FileInputStream("in.dat");
BufferedInputStream bin = new BufferedInputStream(fin);
DataInputStream din = new DataInputStream(bin);
```

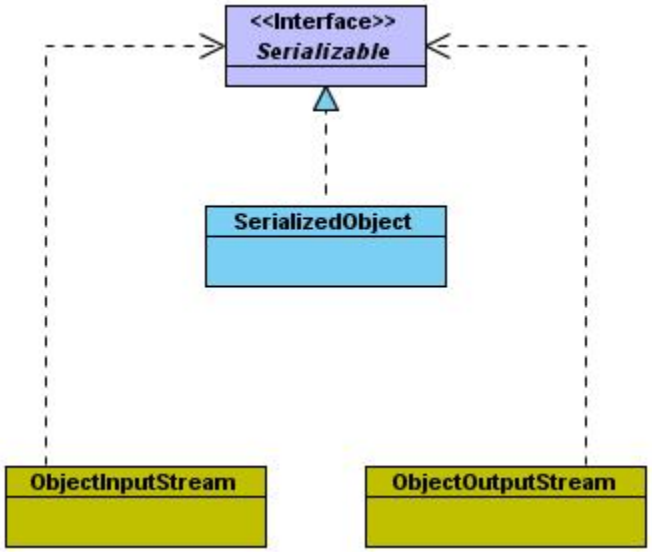
```
DataInputStream in = new DataInputStream(
            new BufferedInputStream(
                new FileInputStream("in.dat")));
```

Serialization

- Serialization is the process of reading and writing objects to a byte stream



Serializable Inteface



- In Java, object that requires to be serialized must implement:
  - **java.io.Serializable** or
  - **java.io.Externalizable** interface
- **Serializable** interface is an *empty* interface (or *tagged* interface) with nothing declared;
- Its purpose is simply to declare that particular object is serializable.

ObjectInputStream class

- ObjectInputStream class extends the InputStream class and implements the ObjectInput interface
- ObjectInput interface extends DataInput interface and has methods that support object serialization
- ObjectOutputStream class extends the OutputStream and implements the ObjectOutput interface
- Important methods:
  - ✓ **ObjectInputStream.readObject(),**
  - ✓ **ObjectOutputStream.writeObject()**

ObjectInputStream class

Save serialized objects to a stream:

```
ObjectOutputStream out = new ObjectOutputStream(
                new BufferedOutputStream(
                        new FileOutputStream("object.dat")));
out.writeObject("The current Date and Time is "); // write a String object
out.writeObject(new Date()); // write a Date object out.flush(); out.close();
```

Load serialized objects from a stream:

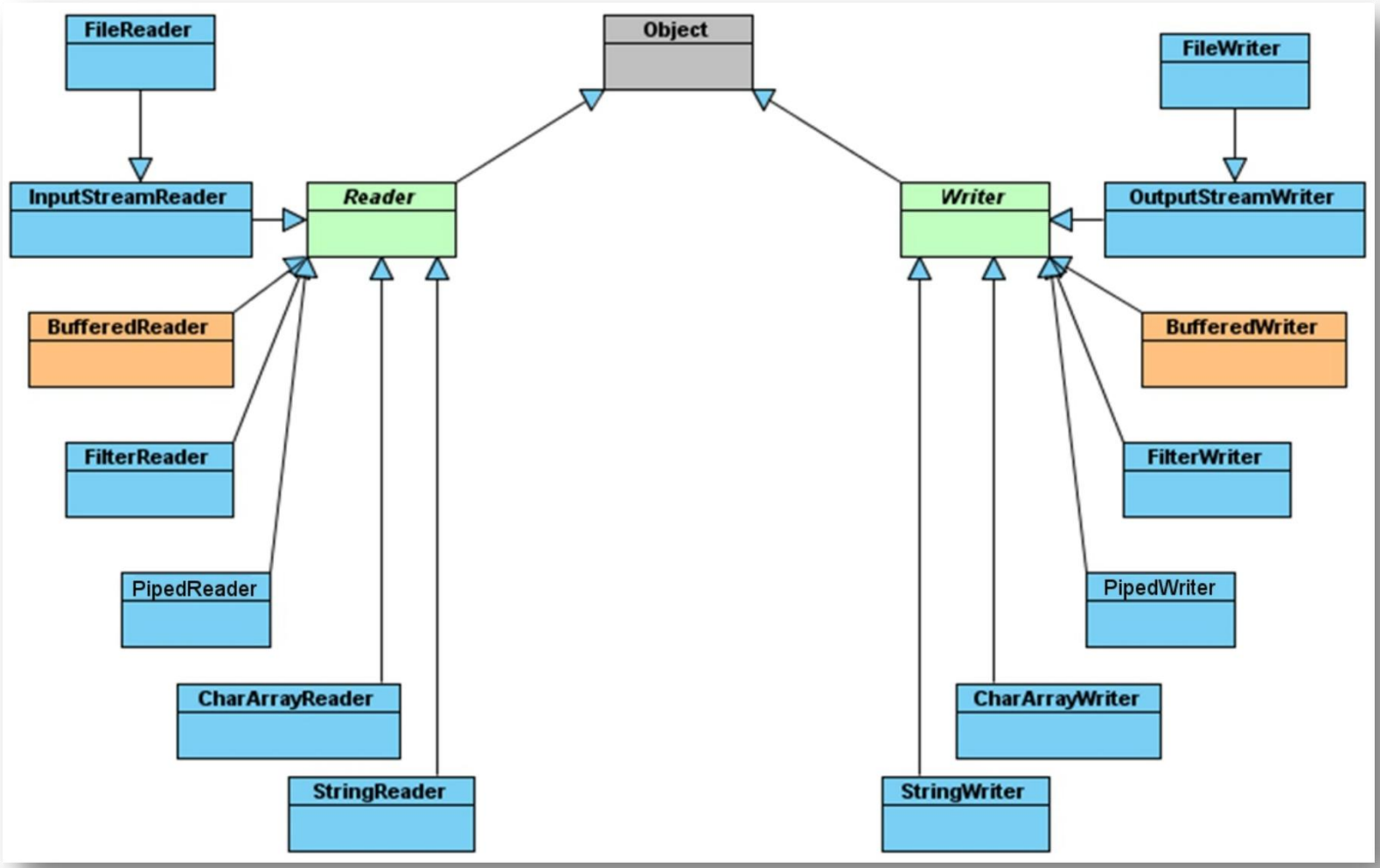
```
ObjectInputStream out = new ObjectInputStream(
                new BufferedInputStream(
                        new FileInputStream("object.dat")));
String str = (String)in.readObject();
Date d = (Date)in.readObject();
in.close();
```

Section 3

CHARACTER STREAMS

java.io package

Character-Based IO & Character Streams



Reader/Writer classes

- Reader class is an abstract class used for reading character streams.
- Writer class is an abstract class and supports writing characters into streams.

Reader/Writer sub-classes

- **CharArrayReader** class is a subclass of Reader class. The class uses character array as the source of text to be read.
- **CharArrayWriter** uses a character array into which characters are written. The methods toCharArray(), toString() and writeTo() method can be used to retrieve the data.

PrintWriter class

- PrintWriter class is a character-based class that is useful for console output. This class provides support for Unicode characters.

```
…
InputStreamReader reader = new InputStreamReader
(System.in);
OutputStreamWriter writer = new OutputStreamWriter
(System.out);
PrintWriter pwObj = new PrintWriter (writer,true);
…
try {
    while   (tmp != -1) {
          tmp   = reader.read ();
          ch = (char) tmp;
          pw.println ("echo " + ch);
        }
     }
     catch   (IOException e) {
          System.out.println ("IO error:" + e );
}
…
```

PrintWriter class

Constructor:

```
PrintWriter(Writer out)
 → Can append
PrintWriter(Writer out, boolean autoFlush)
PrintWriter(OutputStream out)
PrintWriter(OutputStream out, boolean autoFlush)
PrintWriter(String fileName)
 → Cannot append
```

PrintWriter class

- To open a text file for output: connect a text file to a stream for writing

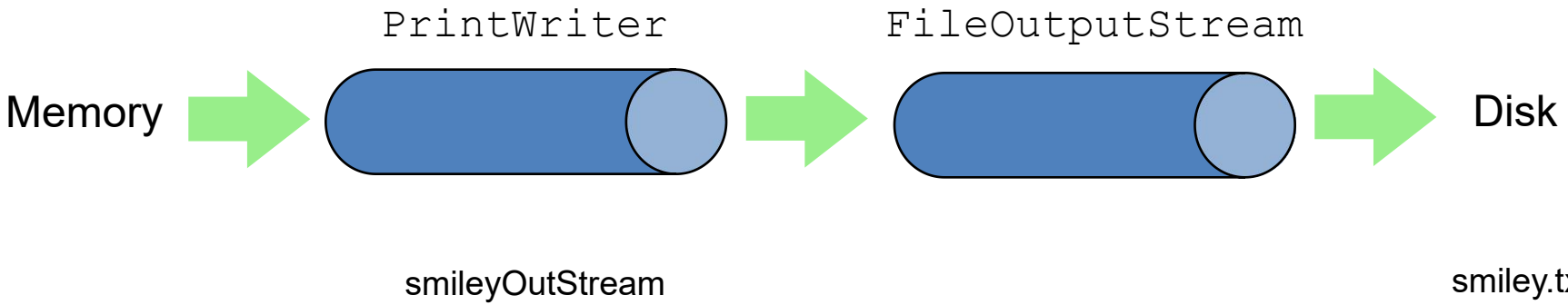
```
PrintWriter outputStream =
        new PrintWriter(new FileOutputStream("out.txt"));
```

- Similar to the long way:

```
FileOutputStream s = new
                        FileOutputStream("out.txt");
PrintWriter outputStream = new PrintWriter(s);
```

- **Goal**: create a `PrintWriter` object:
  - ✓ which uses `FileOutputStream` to open a text file.
- `FileOutputStream` "connects" `PrintWriter` to a text file.

PrintWriter class



PrintWriter smileyOutStream = new PrintWriter(new FileOutputStream(“smiley.txt”));

Reference: **PrintWriterDemo.java**

Java Tip: Appending to a Text File

- To add/append to a file instead of replacing it, use a different constructor for FileOutputStream:

```
outputStream =
new PrintWriter(new FileOutputStream("out.txt", true));
System.out.println("A for append or N for new file:");
char ans = Scanner.next().charAt(0);
boolean append = (ans == 'A' || ans == 'a');
outputStream = new PrintWriter(
new FileOutputStream("out.txt", append));
```

true if user enters 'A'

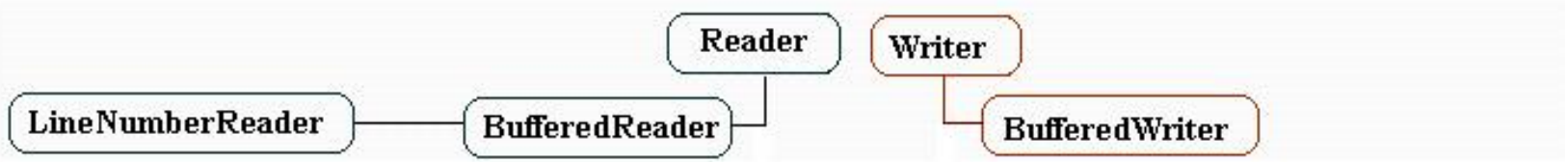
Closing a File

- An output file should be closed when you are done writing to it .
- An input file should be closed when you are done reading from it.
- Use the `close` method of the class `PrintWriter`, `BufferedReader`.

```
outputStream.close();
```

- If a program ends normally it will close any files that are open.

BufferReader class



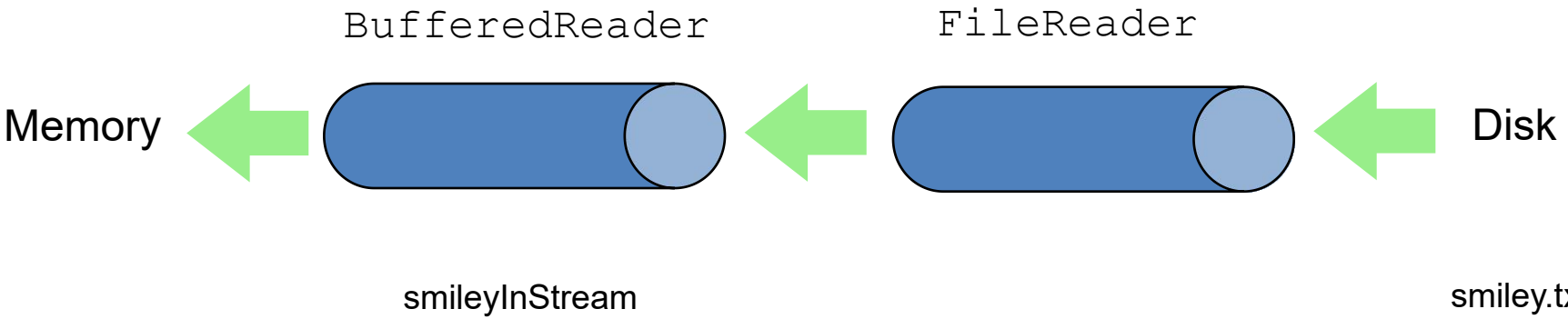
- To open a text file for input: connect a text file to a stream for reading
  - ✓ Goal: a `BufferedReader` object,
    - which uses `FileReader` to open a text file
  - ✓ `FileReader` "connects" `BufferedReader` to the text file
- For example:

```
BufferedReader smileyInStream = new BufferedReader
                        (new FileReader("smiley.txt"));
```

- Similarly, the long way :

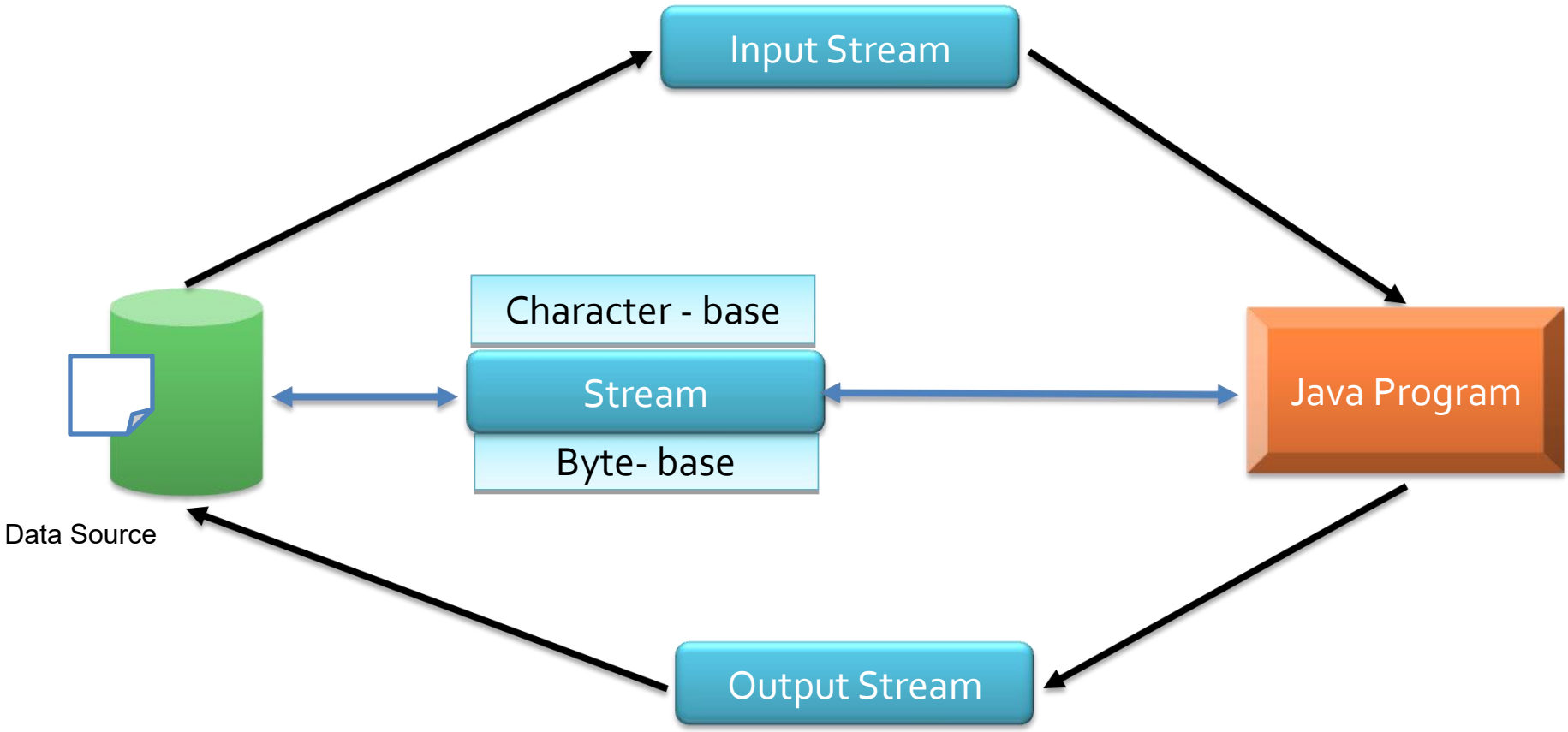
```
FileReader s = new FileReader("smiley.txt");
BufferedReader smileyInStream=new BufferedReader(s);
```

BufferReader class



BufferedReader smileyInStream = new BufferedReader(new FileReader(“smiley.txt”));
Reference: BufferedReaderDemo.java

Summary


Input Stream
Character - base
Stream
Byte- base
Java Program
Data Source
Output Stream

Lesson Summary

- JAVA I/O INTRODUCTION
- FILE
- BINARY STREAMS
  - ✓ DataInputStream/DataOutputStream
  - ✓ FileInputStream/FileOutputStream
  - ✓ ObjectInputStream/ObjectOutputStream
- CHARACTER STREAMS
  - ✓ BufferReader/BufferWriter

Thank you